El internet

David Zelaya
LYFTER

I Parte.

Paso #1

Una vez que el usuario escribe la dirección en el cliente o navegador y presiona la tecla “Enter”, el navegador revisa lo siguiente:

- Cache.
- Cookies/datos de la sesión.

Si el navegador ya conoce el IP, se salta esa parte. De lo contrario, el navegador necesita ubicar la dirección del servidor.

Paso #2

El siguiente paso sería la búsqueda del *DNS(Domain Name System*). Esto es necesario, ya que las computadoras reconocen solamente direcciones IP y no así los nombres de las páginas web.

Una vez que el navegador verifica que la dirección no se encuentra en el cache local, OS cache o en el *router*, el servidor ISP DNS le indica al navegador que www.youtube.com, es la dirección IP

Paso #3

Una vez que el navegador obtiene la dirección IP, ya sabe hacia donde enviar la solicitud. Así que, en este paso, el navegador prepara la solicitud antes de enviarla.

Ejemplo:

GET/HTTP/1.1

Host: www.youtube.com

Paso #4

Ya que YouTube usa HTTPS, parte del siguiente paso sería establecer la conexión TCP(*Transmission Control Protocol*), siguiendo el protocolo de seguridad, que seria el conocido *3-way handshake* (SYN, SYN-ACK, y ACK). Seguido del *TLS handshake*; una vez que el navegador verifica el certificado de seguridad de la página, ambas partes llegan a un acuerdo en cuanto al método de encriptación y los *keys* de seguridad.

Por lo tanto, ahora la sesión debería estar completamente encriptada.

Paso #5

En este paso es cuando el navegador envía formalmente la solicitud HTTP/HTTPS a la página. HTTPS = HTTP + encriptación.

Paso #6

Una vez que la solicitud llega a la infraestructura de la página el servidor la recibe y la dirige internamente para seguidamente determinar:

- Quien es el usuario, esto por medio de cookies/login.
- Que enseñar en pantalla (recomendaciones, página de inicio).

Luego el servidor responde con una infraestructura completa (HTML, CSS, JavaScript).

Paso #7

En este paso el navegador construye la página, usando HTML, CSS y ejecutando el JavaScript.

Paso #8

Seguidamente el navegador envía una serie de solicitudes adicionales.

El API se encarga de la *metadata* del video y las recomendaciones, por lo que el navegador envía solicitudes adicionales para imágenes(*thumbnails*), fuente y scripts.

Paso #9

Una vez que el usuario le da click a un video, el navegador envía otra solicitud. YouTube responde enviando el video en segmentos, no todo a la vez. En esta parte la calidad/ajustes del video dependerán de factores como la calidad del internet.

Paso #10

El video aparece en pantalla, el navegador lo carga por unos segundos/momentos y lo decodifica usando los códecs de hardware/software y finalmente envía los *frames*(cuadros/marcos) a la pantalla.

II Parte.

A) **Frontend vs backend.**

1. **Frontend**:

En el caso de un app web para agendar citas médicas, la parte de **frontend** seria todo con lo que los pacientes o profesionales médicos estarían interactuando directamente. Esto puede incluir partes de la página web como:

- Página de inicio de sesión.
- Calendario para agendar citas.
- Lista de citas agendadas.

Esta parte corre directamente desde el navegador/cliente.

2. **Backend**:

En esta parte es donde ocurre la lógica principal de la aplicación y donde se almacenan datos como:

- Usuarios, ya sean pacientes o profesionales médicos.
- Citas agendadas.
- Disponibilidad para agendar citas.

También maneja la parte de autenticación con el usuario y se encarga de enviar confirmaciones.

Esta parte corre directamente desde el servidor.

B) **Tecnologías que se podrían usar:**

1. Frontend:

- HTML, CSS y JavaScript
- React
- Vue.js

2. Backend:

- Node.js
- Django
- Spring Boot

C) Como se comunica frontend con backend:

En esta parte es cuando el API + HTTP + *request/response*, hacen conexión.

Paso 1:

- En el frontend, el usuario selecciona la opción de agendar una cita.
- Ejemplo:
  -Fecha: 3 de marzo del 2025
  -Hora: 10:00am
  -Ortopedia

Paso 2:

Frontend envía una solicitud al backend, usando un API.

Ejemplo: **POST /api/ citas**

El frontend envía la información en un archivo JSON:

```
{
"consultorio": "Ortopedia",
"fecha": "2025-03-03",
"hora": "10:00"
}
```

Esto es lo que se conoce como *HTTP request*(solicitud).

Paso 3:

El backend recibe la solicitud y la procesa. Al procesarla, revisa:

- Si el doctor del respectivo consultorio está disponible a la hora seleccionada.
- Que no haya otra cita agendada previamente para esa misma hora.

Una vez se valida esta información, la misma es almacenada en la base de datos.

Paso 4:

En este paso, backend envía una respuesta, más conocida como *HTTP response*.

En caso de éxito:

{

“status”: “confirmada”,

“numero_confirmacion”: KFJSOSM123,

}

En caso de error:

{

“error”: “Hora no disponible”

}

Paso 5:

Frontend actualiza el *UI*. En caso de agendar la cita con éxito, se mostrará una breve confirmación en pantalla. En caso de error, el usuario recibirá un mensaje en pantalla detallando lo ocurrido.

Conceptos claves

A) API:

En este caso sería el menú de lo que backend permite hacer.

Ejemplo:

- POST /citas
- GET /citas
- DELETE /citas/{numero_confirmacion}

B) HTTP:

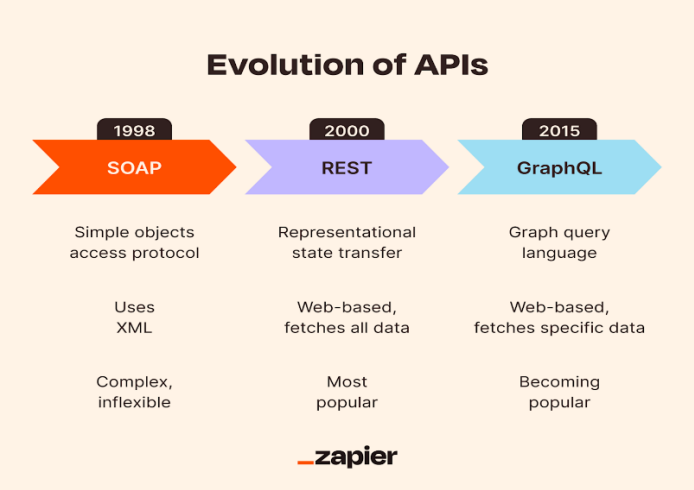
El protocolo usado para comunicación. Define:

- Métodos (GET, POST, PUT, DELETE)
- Encabezados(*headers*)
- Código de *status* (200, 404, 500)

C) Request/response:

El ciclo se vería de la siguiente manera:

- Request: *frontend -> backend*
- Response: *backend -> frontend*



III Parte

Tipo de API	Formato de datos usado	Nivel de flexibilidad	Dificultad de implementación	Uso actual (Alta / Media / Baja)
REST	Principalmente JSON, en ocasiones XML	Mediano: el servidor define la estructura, el navegador/cliente define el punto final(endpoint)	Baja-mediana	Alto
SOAP	Solo XML	Bajo: el servidor define todo exclusivamente	Alta	Bajo(legacy/enterprise)
GraphQL	JSON	Alto: el cliente/navegador, tiene control total sobre qué información recibe.	Mediana-alta	Mediano-alto (en crecimiento)

¿Cuál considera más apropiada para una *startup* moderno que desarrolla un sistema de reservas en línea? ¿Por qué?

En este caso, la opción por default o la más cómoda para un *startup* seria *REST*. Ya que:

- Se construye rápidamente.
- Es más sencillo realizar *debug*.
- Vasto ecosistema.
- Todo desarrollador profesional tiene/debería tener el conocimiento y capacidad de entenderlo, ya que es sencillo, predecible y fácil de adaptar cuando se empieza un proyecto.

Respecto a las otras 2 opciones de API, *GraphQL* podría ser una mejor opción en caso de que la aplicación tuviese múltiples *frontends* (*web* + *mobile* + *admin*), ya que cuenta con una capacidad de obtención de datos más precisa, ya que en un solo *request* puede obtener mayor cantidad de información. En el caso de REST,

estaríamos hablando de múltiples *requests*, lo que hace la obtención de datos sea menos dinámica.

En el caso de *SOAP*, no sería una buena opción para un sistema de *startup* de reservaciones, ya que:

- Se tarda mucho tiempo en construir
- Mucha rigidez.
- El servidor define todos los detalles (estructura, reglas, formato de datos usado) lo cual restringe casi por completo la autoridad del cliente/navegador.

Este API seria una buena opción en caso de tener que estructurar una aplicación bancaria o algún otro sistema *legacy*, que solo sea compatible con esta última opción.

IV Parte

Parte 1: Selección de API

Nombres:

- *Squiggle API:* Ofrece: retorna una lista de fuentes de datos relacionadas al *football* de reglas australiano (AFL), con datos como tabla, marcadores, fechas de encuentros y demás, por medio de un archivo JSON.
- *JSON placeholder*: Datos falsos para crear tests y hacer prototipos.

Parte 2/3: Configuración en Postman/Ejecución y análisis

A) Códigos de estado:

Solicitud/*Request*	**Códigos de estado**	**Resultado**
Get data	200 OK	La solicitud se realizó con éxito.
Get Games	200 OK	La solicitud se realizó con éxito.
Post data	201 Created	El recurso fue creado.
Update data	200 OK	El recurso fue actualizado exitosamente.
Delete post	200 OK	El recurso fue eliminado exitosamente.

B) Cuerpo de la respuesta (JSON o texto):

- Solicitudes GET (Squiggle API):

Get data:


```
Body  Cookies  Headers 12  Test Results 1/1  200 OK  313 ms
{} JSON  Preview  Visualize
1  {
2      "sources": [
3          {
4              "url": "https://live.squiggle.com.au/",
5              "icon": "/wp-content/themes/squiggle/assets/images/Squiggle.png",
6              "id": 1,
7              "name": "Squiggle"
8          },
```


Get Games:


```
Body  Cookies  Headers 12  Test Results  200 OK
{} JSON  Preview  Visualize
1  {
2      "games": [
3          {
4              "is_final": 0,
5              "year": 2020,
6              "updated": "2020-03-19 22:02:03",
7              "date": "2020-03-19 19:40:00",
8              "round": 1,
9              "hgoals": 16,
10             "agoals": 12,
11             "localtime": "2020-03-19 19:40:00",
```


La respuesta del cuerpo/*body* aparece en formato JSON, por lo que contiene una serie de datos estructurados como listas de fuentes y recursos relacionados a estadísticas y resultados de encuentros/partidos anteriores.

Solicitud POST:

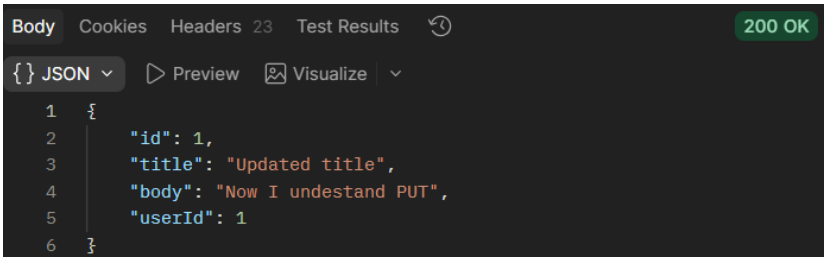

```
Body  Cookies  Headers 25  Test Results 1/1  201 Created
{} JSON  Preview  Visualize
1  {
2      "title": "Practice Post",
3      "body": "Learning Postman step by step",
4      "userId": 1,
5      "id": 101
6  }
```


El cuerpo/*body* de la respuesta, retorna el objeto creado y genera un número de identificación.

En este caso, **"id": 101**

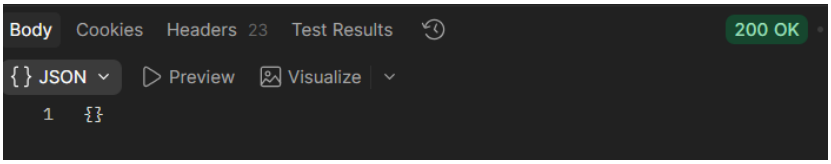
-Como se aprecia en la imagen del resultado.

Solicitud PUT:



En la respuesta, el cuerpo refleja la actualización en el campo/espacio modificado.

Solicitud DELETE:



La respuesta refleja un cuerpo vacío, indicando que el recurso fue eliminado exitosamente.

C) Encabezados/*headers*:

- **Content-Type**
  application/json; charset=UTF-8 (mismo en todos los requests)

- **Authorization**
  No hay ningún *header* de *Authorization*, ya que los *APIs* seleccionados no requieren autenticación (tal y como lo solicitan las instrucciones).

- **Content-length**
  Post data: 101
  Delete post: 2

- **Date**
  Mon, 30 Mar 2026 00:41:47 GMT (mismo en todos los requests)

Códigos de estado: Todas las solicitudes/*requests*, fueron realizadas de manera exitosa. La solicitud POST retorno el código *201 Created*, confirmando así, la creación de el nuevo objeto. Mientras que todas demás solicitudes retornaron el código *200 - OK*, indicando que la ejecución fue exitosa.

Cuerpo de respuesta: Todas las solicitudes retornaron respuestas en formato JSON. Las solicitudes GET, retornaron datos estructurados como recursos y estadísticas de encuentros de la liga australiana. La solicitud PUT, retorno un nuevo objeto y la solicitud DELETE, retorno un cuerpo vació, indicando de esta manera que la solicitud fue exitosa.

Headers: Las respuestas contienen *headers*/encabezados como *Content-Type*, confirmando así que los datos se encuentran en formato JSON. No se encontró ningún *header* de *Authorization*, ya que los *APIs* utilizados no requieren autenticación como tal. También se agregó información acerca de *headers* como *Date* y *Content-Lenght* (para *POST* y *DELETE* solamente).

Parte 4:

(Leer el archivo README.md)

Parte 5:

- **¿Qué aprendiste sobre el funcionamiento de las APIs?**

  Aprendimos que los *APIs* funcionan como un puente que permite la comunicación entre diferentes sistemas por medio del envió de solicitudes, posteriormente se recibe una respuesta, por lo general en formato JSON. Cada API presenta *endpoints* que representan fuentes/recursos, los cuales son acezados usando métodos HTTP como GET, POST, PUT y DELETE. Estos corresponden a la metodología *CRUD (Create, Read, Update, Delete)*. También aprendimos como los parámetros pueden ser usados para filtrar datos y como las respuestas incluyen códigos de estado/status, *headers* y un cuerpo que juntos describen el resultado de la solicitud.

- **¿Cómo te ayudó Postman a entender la comunicación entre cliente y servidor?**

  Postman me ayudó a entender la comunicación entre el cliente y el servidor, permitiéndome crear y enviar solicitudes HTTP manualmente y recibir una respuesta de manera inmediata. Trabajando con diferentes métodos como GET, POST, PUT y DELETE, pude ver de manera clara y concisa como el navegador/cliente, envía datos al servidor y como este procesa la solicitud y responde con un código de status/estado, *headers* y datos en formato JSON. Este acercamiento me hizo entender cómo opera la metodología CRUD en tiempo real y el rol que tiene cada solicitud y respuesta respecto a cada interacción como tal entre sistemas.